



## Chapter 2 – Software Processes

# Topics covered

---



- ❑ Software process models

- ❑ Process activities

- ❑ Coping with change

- ❑ Process improvement

# The software process

---



## ➤ What

- set of activities required to develop software.

## ➤ Activities:

- Specification (What)
- Design and implementation (How)
- Validation – checking that it does what the customer wants;
- Evolution – changing the system in response to changing customer needs.

## ➤ Model

- abstract representation of a process.
- describes process from perspective.

# The software process types

---



- Plan-driven processes
  - all of the process activities are planned in advance
  - progress is measured against this plan.
- Agile processes
  - planning is incremental
  - easier to change the process to reflect changing customer requirements.
- Hybrid
  - Practically, most processes include mixture of both types



# Software process models

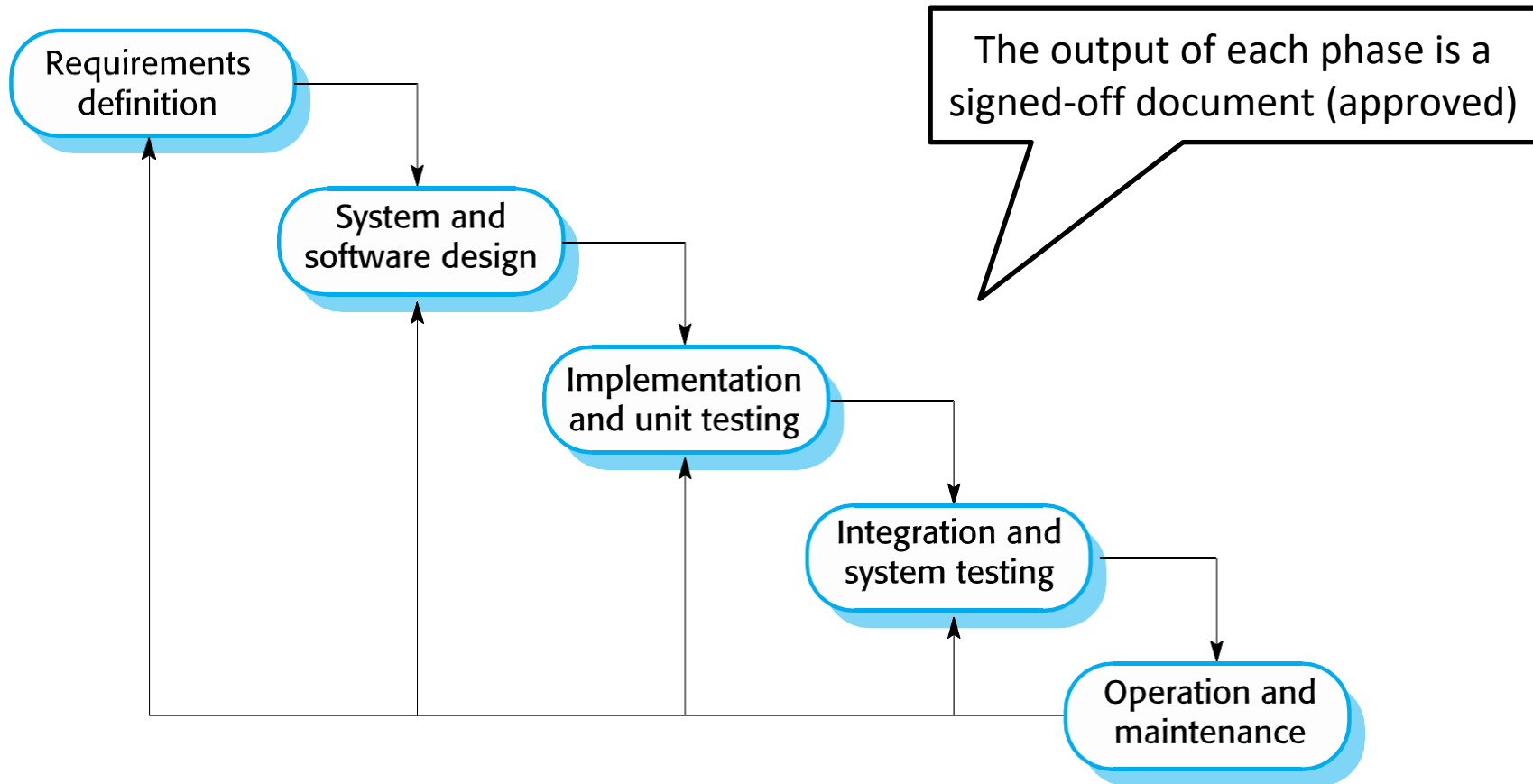
# Process models

---



- The waterfall model
  - Plan-driven model.
  - Phases are separated
- Incremental development
  - Specification, development and validation are interleaved.
  - Plan-driven or agile.
- Integration and configuration
  - The system is assembled from existing configurable components.
  - Plan-driven or agile.
- Practically?

# The waterfall model phases



# Waterfall model problems

---



## ❑ Drawbacks?

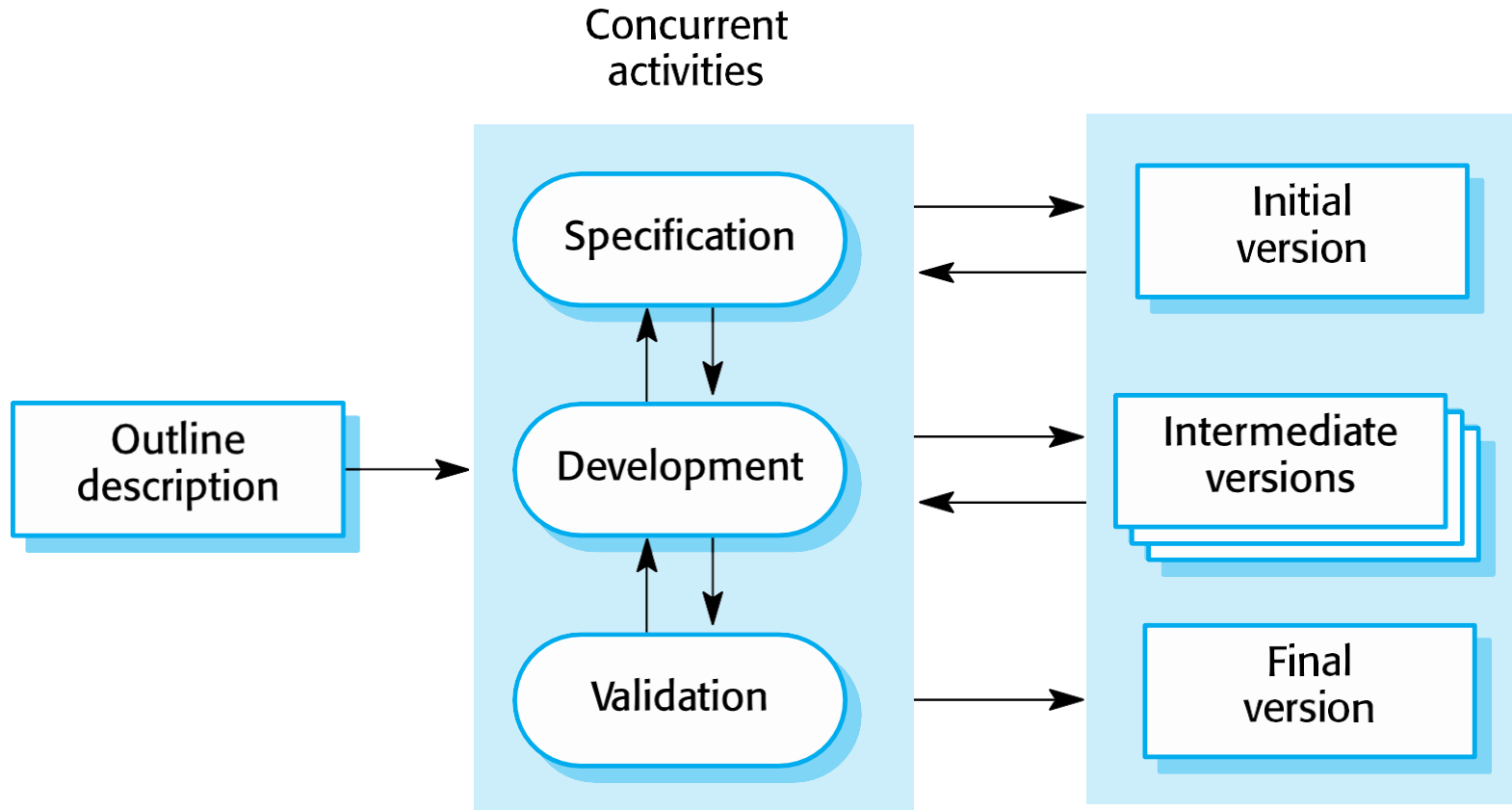
- difficulty to adapt to changes
- Sequential phases
- Phases takes long to execute

## ❑ Usage

- Appropriate when the requirements are well-understood
- Appropriate when changes are limited
- Appropriate for large critical systems & large systems



# Incremental development



# Incremental development

---



## ➤ Pros

- cost of requirement changes is reduced.
- The amount of analysis and documentation that has to be redone is much less than that in waterfall model.
- Easy to get customer feedback
- Rapid delivery
- Working software

# Incremental development

---



## ➤ Cons

- The process is not visible. Managers always need deliverables, not documentation
- Documentation is costly if system is developed quickly
- System structure tends to degrade with new increments.
  - Unless time and money are spent on refactoring
- Inefficient for large, complex and long-lived projects

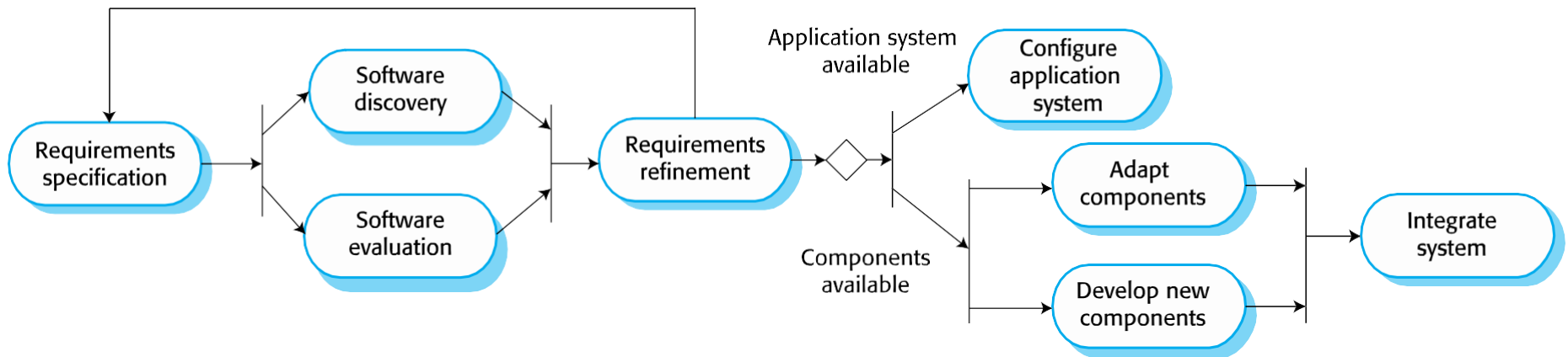
# Integration and configuration

---



- Based on software reusability
  - Don't reinvent the wheel
- Integration
  - COTS: Stand-alone application systems
  - Package/library
  - Web services
- Configuration
  - Adapt functions to customer requirements

# Reuse-oriented software engineering



# Reuse-oriented software

---



## ➤ Pros

- Less software is developed
- Reduced cost
- Faster delivery and deployment

## ➤ Cons

- New requirements (CR) are inevitable
- Loss of control over evolution of reused system elements
- High learning curve for enterprise components
- Tracking system versions for updates
- Integration bugs



# Process activities

# Process activities

---



❑ Inter-leaved sequences of technical, collaborative and managerial activities

- Specification
- Development
- Validation
- Evolution



# Software specification (requirement engineering)

---



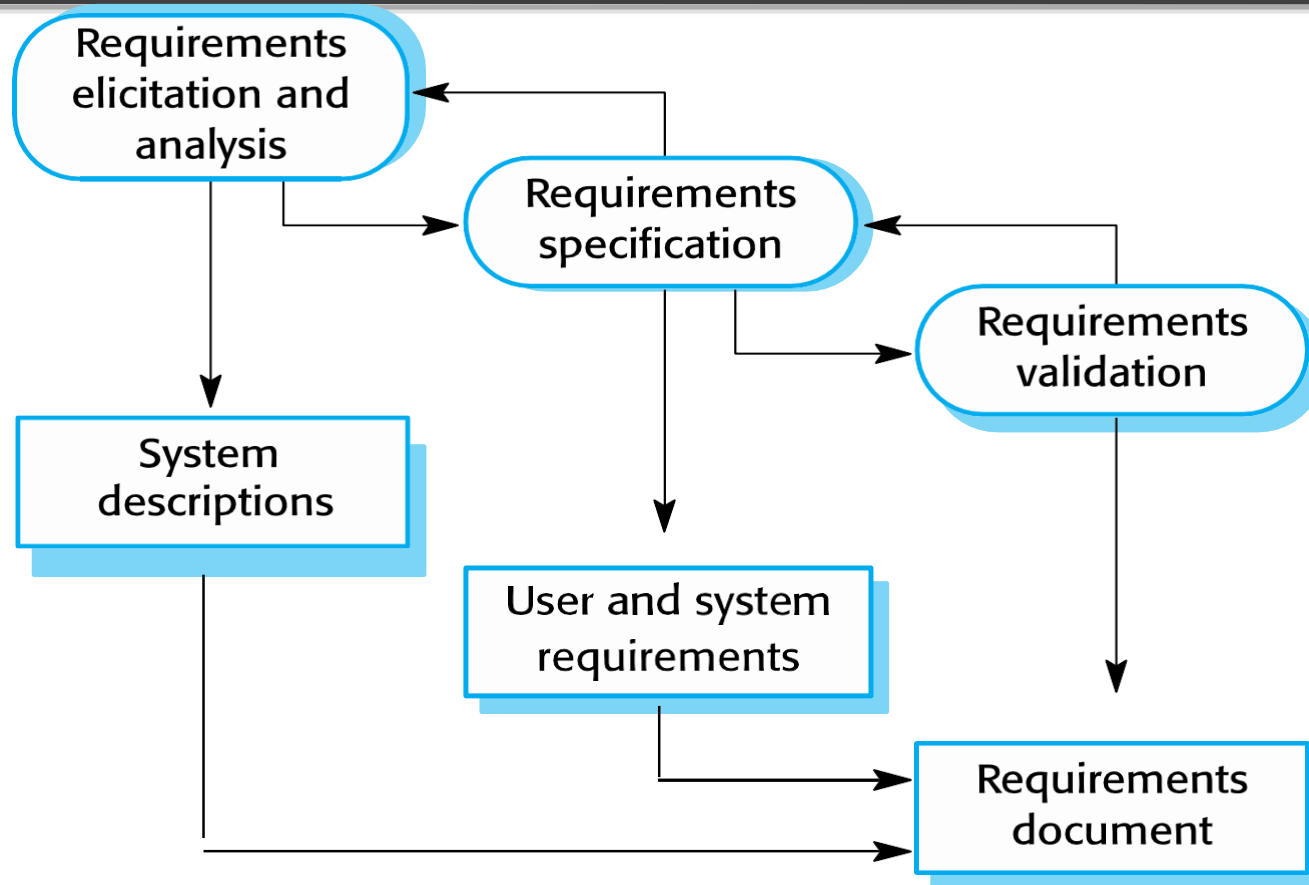
## ➤ What

- services are required?
- constraints are there (operational & development)

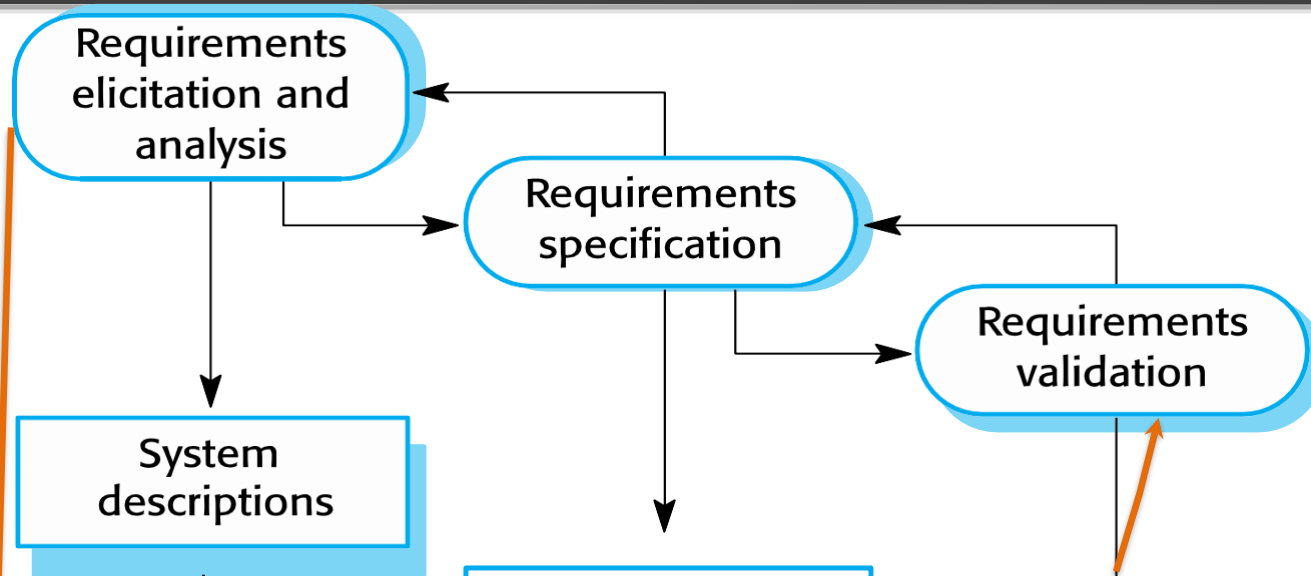
## ➤ Steps (process)

- Pre
  - Feasibility study
  - Market survey
- Requirements elicitation and analysis
  - What do the system stakeholders require or expect from the system?
- Requirements specification
  - Defining the requirements in detail
- Requirements validation
  - Checking the validity of the requirements

# The requirements engineering process



# The requirements engineering process



checks the requirements for **realism**, **consistency**, and **completeness**. During this process, errors in the requirements document are inevitably discovered

may involve the development of one or more system models and prototypes

# Software design and implementation

---



## ➤ What

- converting specs to exe.

## ➤ Design

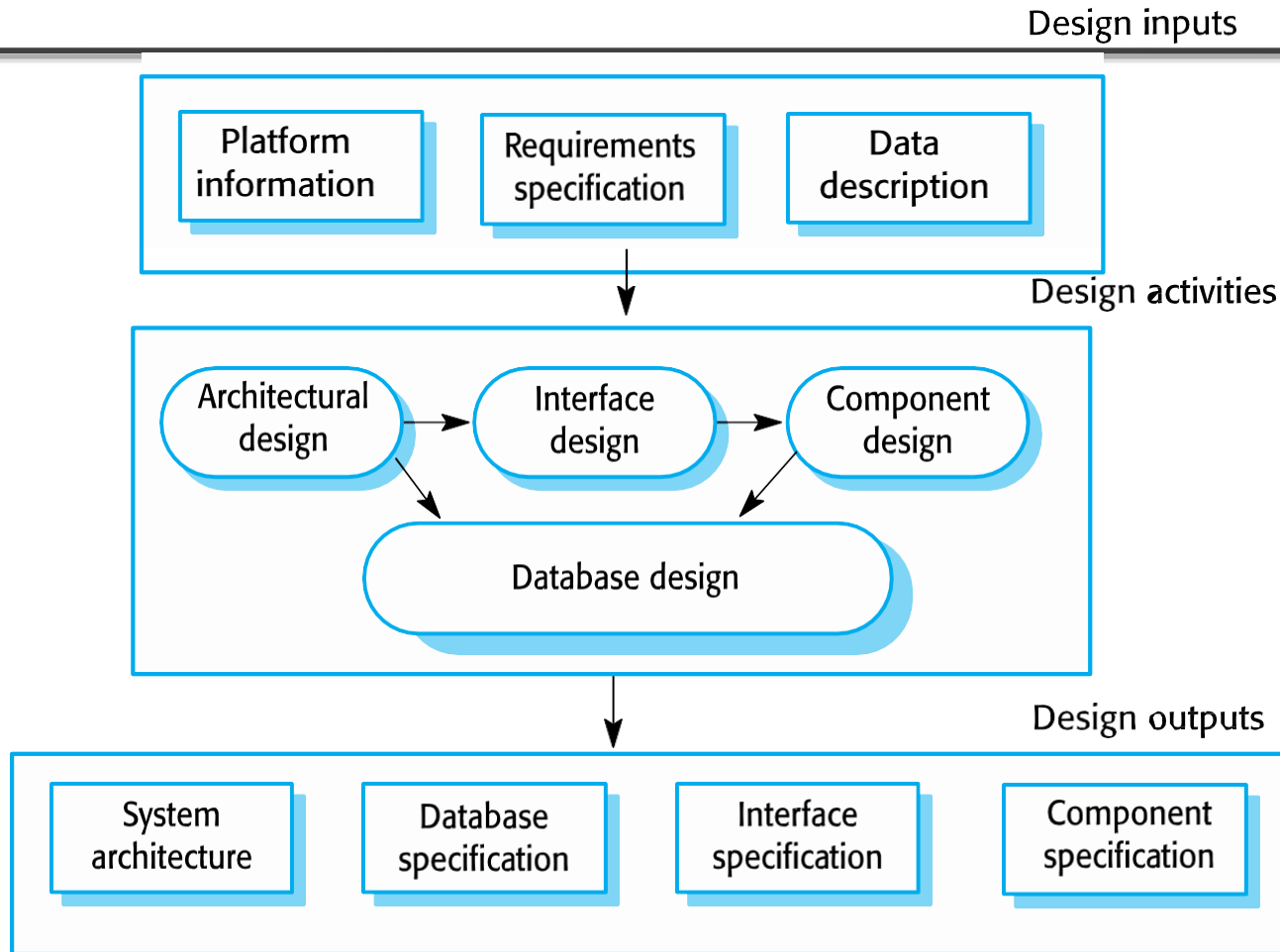
- Design a software structure that realises the specification;

## ➤ Implementation

- Translate this structure into an executable program;

## ➤ Interleaved activities

# A general model of the design process



**if an agile approach to development is used, *design and implementation are interleaved, with no formal design documents produced during the process***

# Design activities

---



## ✧ ***Architectural design***

- overall structure of the system,
- principal components (subsystems or modules),
- relationships & distribution.

## ✧ ***Database design***

- data representation in DB

## ✧ ***Interface design***

- contract implementation

## ✧ ***Component selection***

- search for reusable components.
- design reusable components

# System implementation

---



- by developing a program
- by configuring an application system.
- Programming is an individual activity with no standard process.
- Debugging is the activity of finding program faults and correcting these faults.

# Software validation

---

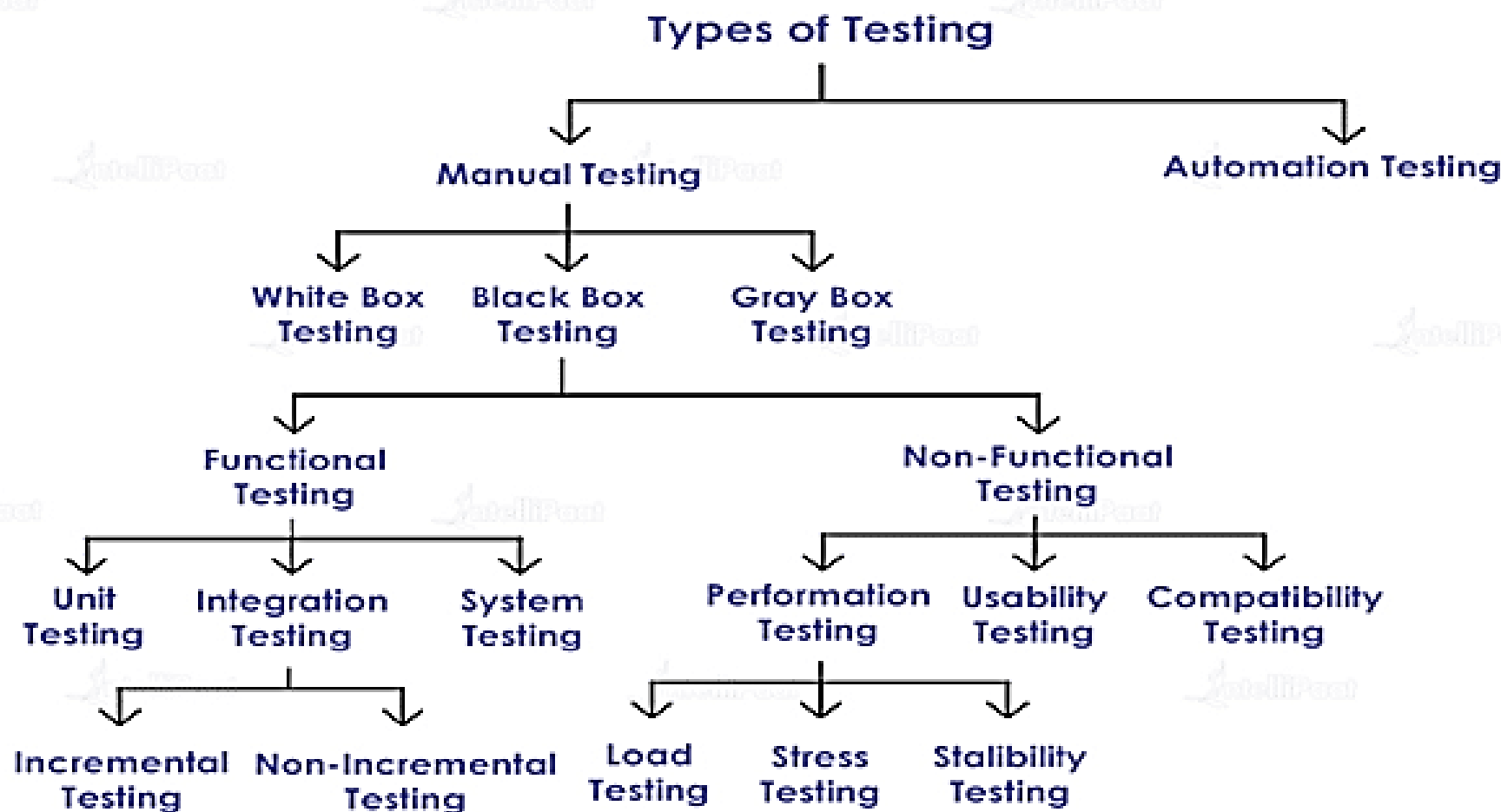


- ✧ Verification and validation (V & V)
  - system conforms to its specification and
  - meets the requirements of the system customer.
- ✧ **Activities**
  - System Checking
  - Process review
  - Testing
- ✧ Testing is the most commonly used V & V activity.

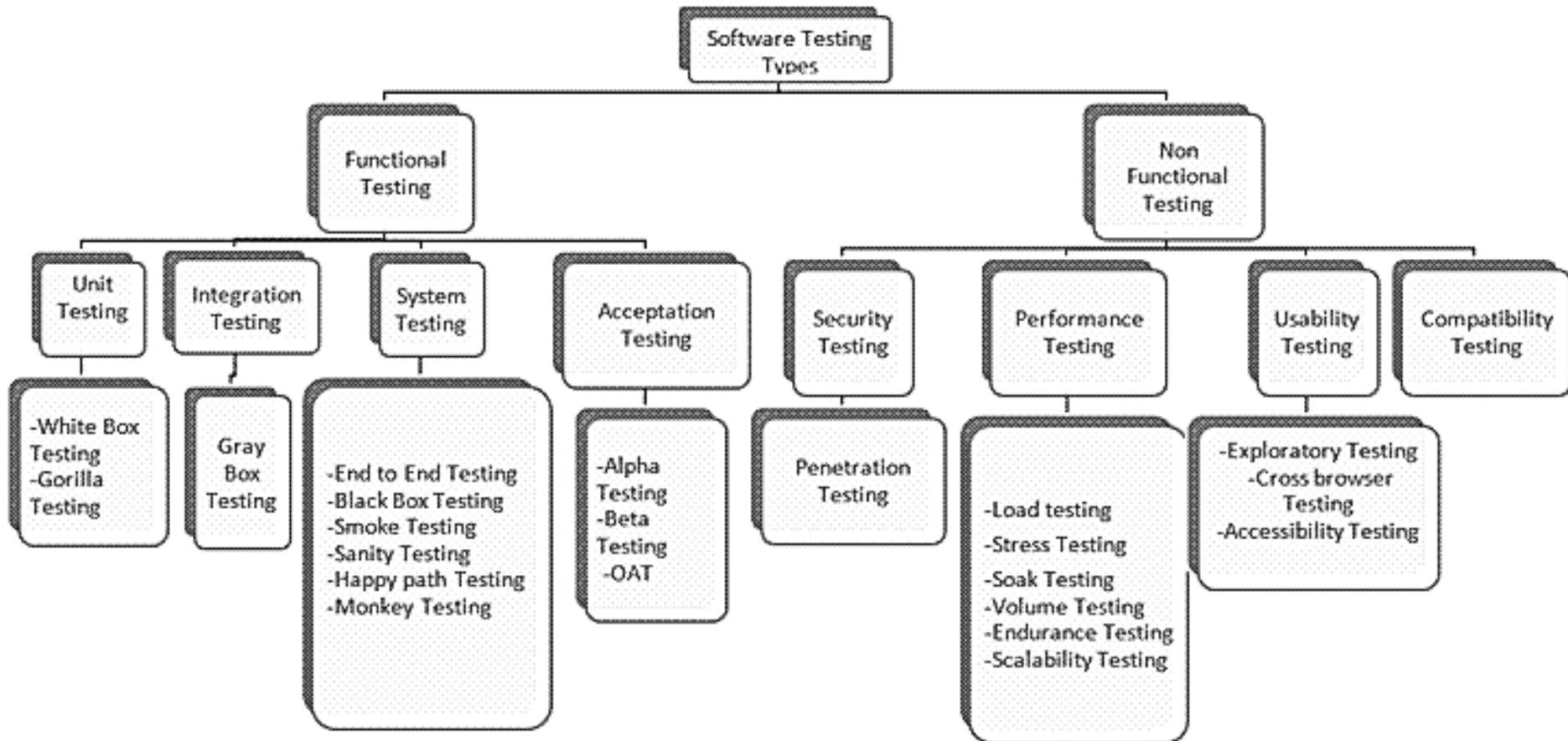




# Software testing types



# Manual software testing



# Functional testing

---



## ❑ Unit testing

- Unit testing is done on an individual unit or component to test its correctness. Unit testing is done by the developer. Each unit can be viewed as a method, function, procedure, or object. Developers use test automation tools such as JUnit for the test execution.

## ❑ Integration Testing

- two or more modules of an application are logically grouped together and tested as a whole. The focus of this type of testing is to find the defect on interface, communication, and data flow among modules

# Functional testing

---



## ❑ System testing

- **End to end testing:** testing a complete application environment to mimic real-world use, such as interacting with a database, using network communications, or interacting with other hardware, applications, or systems if appropriate
- **Smoke testing:** is performed to verify that basic and critical functionality of the system is working fine at a very high level, **whenever a new build** is provided by the development team
- **Sanity testing:** is performed on a system to verify that newly added functionality or bug fixes are working fine. Sanity testing is done on stable build. It is a subset of the regression test
- **Monkey testing:** is to check if an application or system gets crashed by providing random input values/data. Monkey Testing is performed randomly

# Functional testing

---



## ❑ Acceptance testing

- **Alpha testing:** a type of acceptance testing performed by the team in an organization to find as many defects as possible before releasing software to customers
- **Beta testing (aka UAT):** a type of software testing which is carried out by the clients/customers. It is performed in the Real Environment before releasing the product to the market for the actual end-users

# Non-Functional testing

---



## ❑ Penetration testing

- Pen testing is the type of security testing performed as an authorized cyberattack on the system to find out the weak points of the system in terms of security
- Pen testing is performed by outside contractors, generally known as ethical hackers. That is why it is also known as ethical hacking
- Contractors perform different operations like SQL injection, URL manipulation, Privilege Elevation, session expiry, and provide reports to the organization

# Non-Functional testing

---



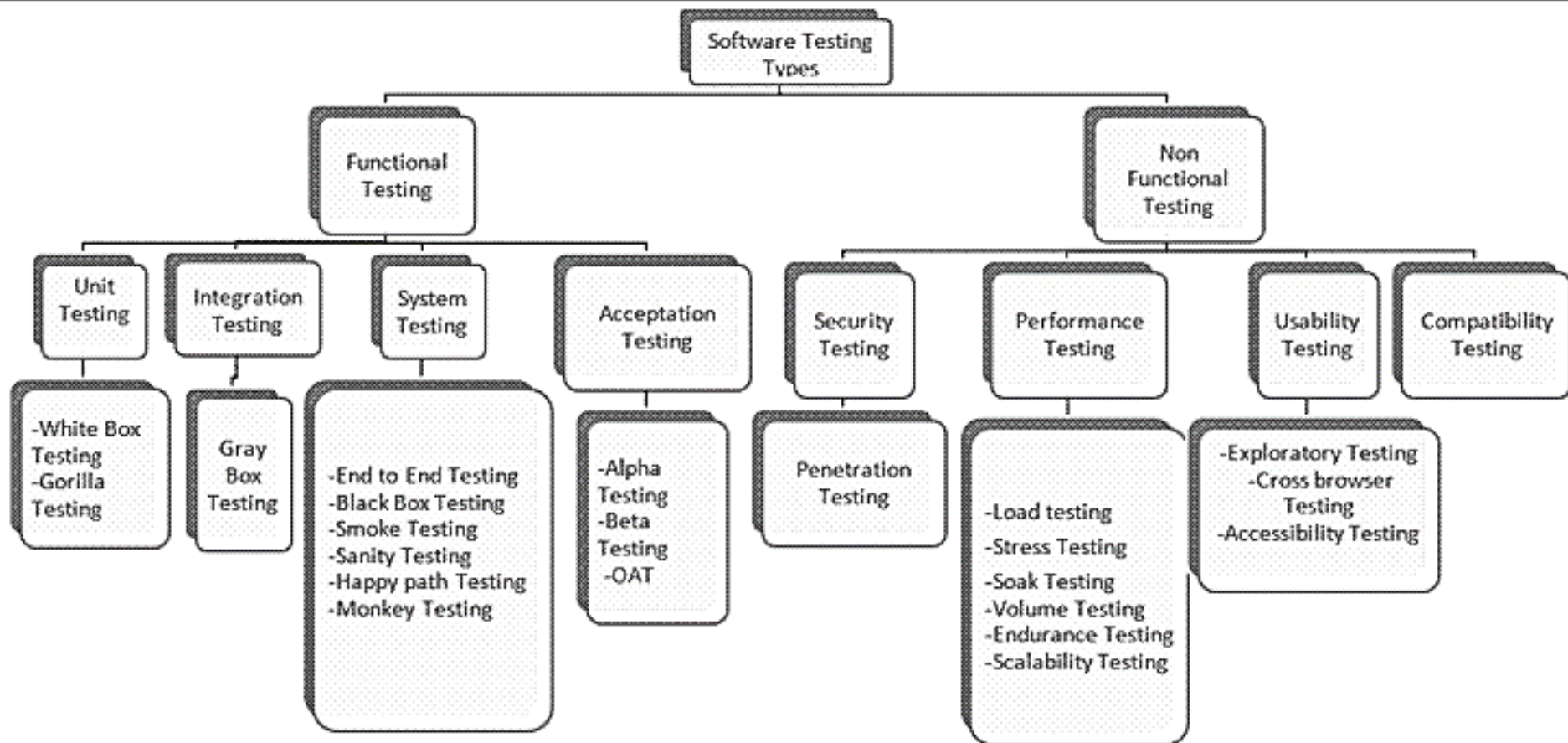
## □ Performance testing

- **Load testing:** testing of an application's stability and response time by applying load, which is **equal to or less than the designed number of users** for an application
- **Stress testing:** testing an application's stability and response time by applying load, which is **more than the designed number of users** for an application
- **Scalability testing:** same as stress testing, but increasing load gradually until finding the avalanche point of the system

Let's say my application is giving response time as follows:

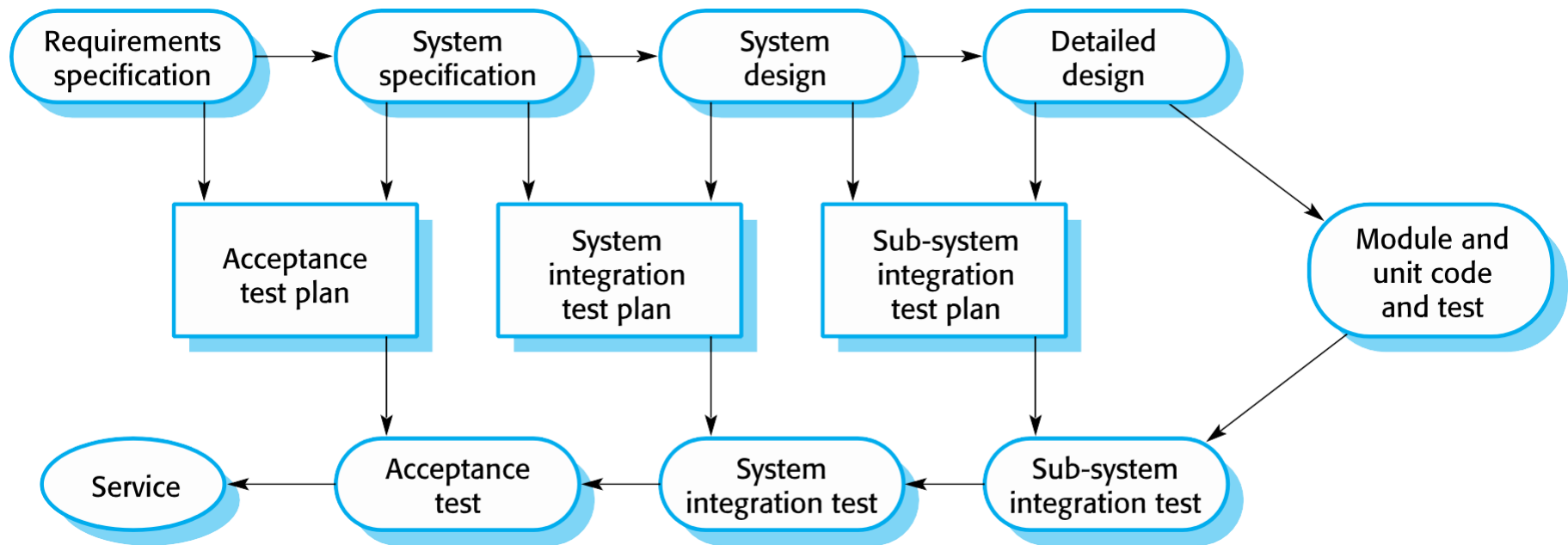
- 1000 users -2 sec
- 1400 users -2 sec
- 4000 users -3 sec
- 5000 users -45 sec
- 5150 users- crash – This is the point that needs to identify in scalability testing

# Usability & Compatibility testing





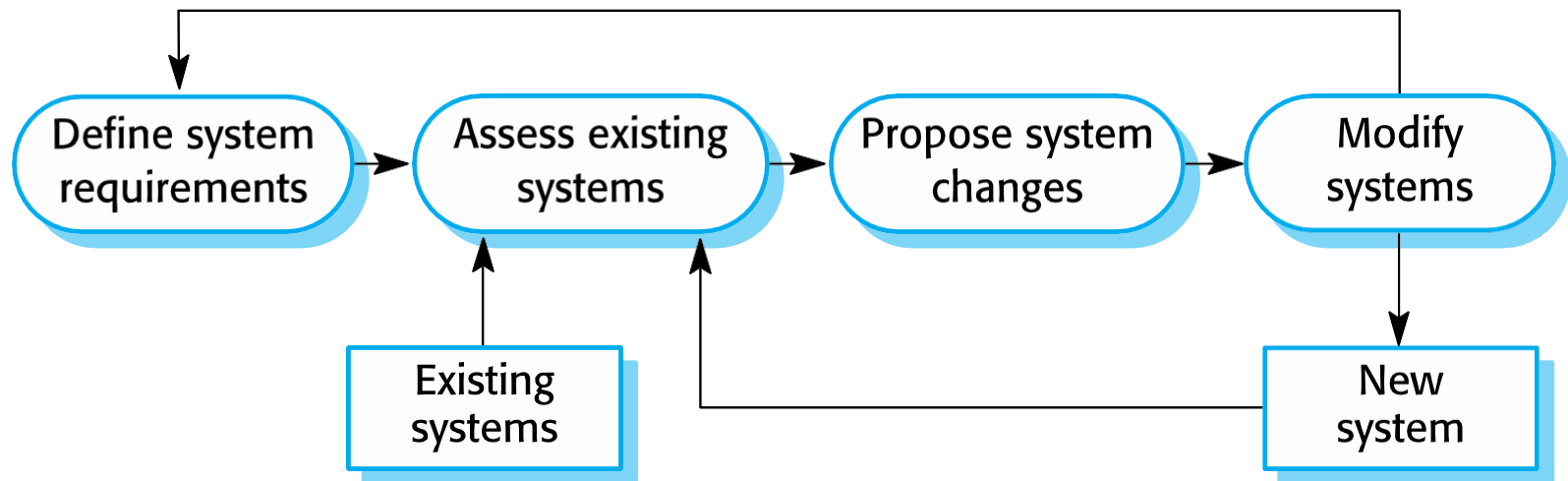
# Testing phases in a plan-driven software process (**V-model**)



# System evolution



CR (customer request / requirement change) → software should evolve





# Coping with change

# Coping with change

---



✧ Change is inevitable in all large software projects.

- Business changes
- New technologies
- Changing platforms

✧ Change → rework → cost

- re-analysing requirements
- re-implementing new functionality

# Reducing the costs of rework



## ✧ Change anticipation.

- By experience
- Community trends
- Similar products
- Flexible architecture
- System prototype for next project phase

## ✧ Change tolerance

- Adopt process that welcomes changes at low costs
- Incremental delivery
  - Change avoidance and tolerance
  - Prevents premature commitment to requirements

# Software prototyping



## ✧ What

- initial version of a system used to demonstrate concepts and try out design options.

## ✧ Usage:

- The requirements engineering process to help with requirements elicitation and validation;
- In design processes to explore options and develop a UI design;
- In the testing process to run back-to-back tests.

# Benefits of prototyping

---



- ✧ Improved system usability.
- ✧ A closer match to users' real needs.
- ✧ Improved design quality.
- ✧ Improved maintainability.
- ✧ Reduced development effort.
- ✧ Reduced cost of changes

# Prototype development

---



- ✧ May be **based** on **rapid prototyping** languages or tools
- ✧ May involve **leaving** out **functionality**
  - Prototype should focus on areas of the product that are not well-understood;
  - Error checking and recovery may not be included in the prototype;
  - Focus on functional rather than non-functional requirements such as reliability and security



# Throw-away prototypes



## ✧ **Prototypes** should be **discarded after development**

- Not good basis for a production system:
- It may be impossible to tune the system to meet non-functional requirements;
- Normally undocumented;
- Structure is usually degraded through rapid change;
- Probably will not meet normal organizational quality standards.

# Incremental development and delivery

---



- ❑ Development and delivery are broken down into **increments**
- ❑ Each increment delivering part of the required functionality.
- ❑ User requirements are **prioritized**
- ❑ Highest priority requirements are included in **early** increments.
- ❑ Requirements are **frozen** once development is underway.

# Incremental development and delivery

---



- ❑ Evaluate each increment before proceeding to the development of the next increment
- ❑ Evaluation is done by user/customer proxy
- ❑ Normally used in agile methods
- ❑ More realistic evaluation about practical use of software
- ❑ Difficult to implement for replacement systems



# Process improvement

# Process improvement

---



## ✧ Why

1. Enhancing software quality
2. Cost reduction
3. Development acceleration

✧ SPI means **understanding** existing **processes** and **changing** these **processes** to meet the items above

# Approaches to improvement

---



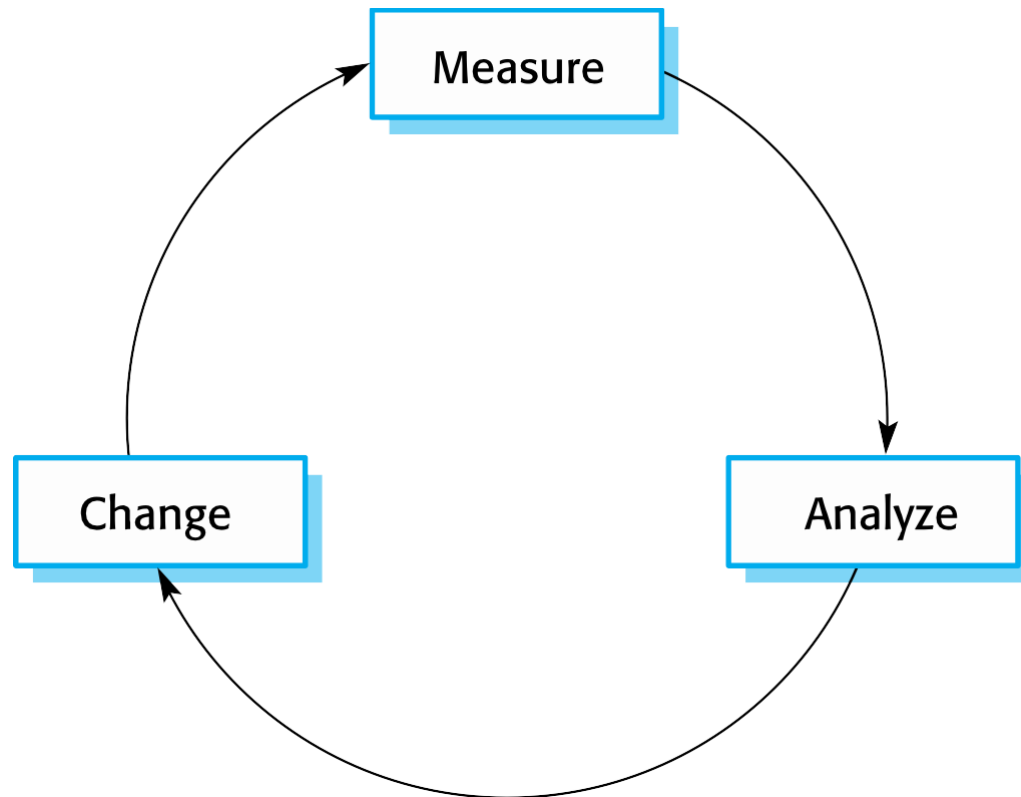
## ✧ *CMM approach*

- focuses on improving **process** and **project management** and introducing **good software engineering practice**.
- The level of process maturity reflects the extent to which good technical and management practice has been adopted in organizational software development processes.

## ✧ Agile approach

- focuses on **iterative development** and the **reduction** of overheads in the software process.
- The primary characteristics of agile methods are rapid delivery of functionality and responsiveness to changing customer requirements.

# The process improvement cycle



# Process improvement activities



## ✧ *Process measurement*

- You measure one or more attributes of the software process or product. These measurements forms a **baseline** that helps you decide if process improvements have been effective.

## ✧ *Process analysis*

- The current process is assessed, and process weaknesses and bottlenecks are identified. Process models (sometimes called process maps) that describe the process may be developed.

## ✧ *Process change*

- Process changes are proposed to address some of the identified process weaknesses. These are introduced and the **cycle resumes to collect data about the effectiveness of the changes**.

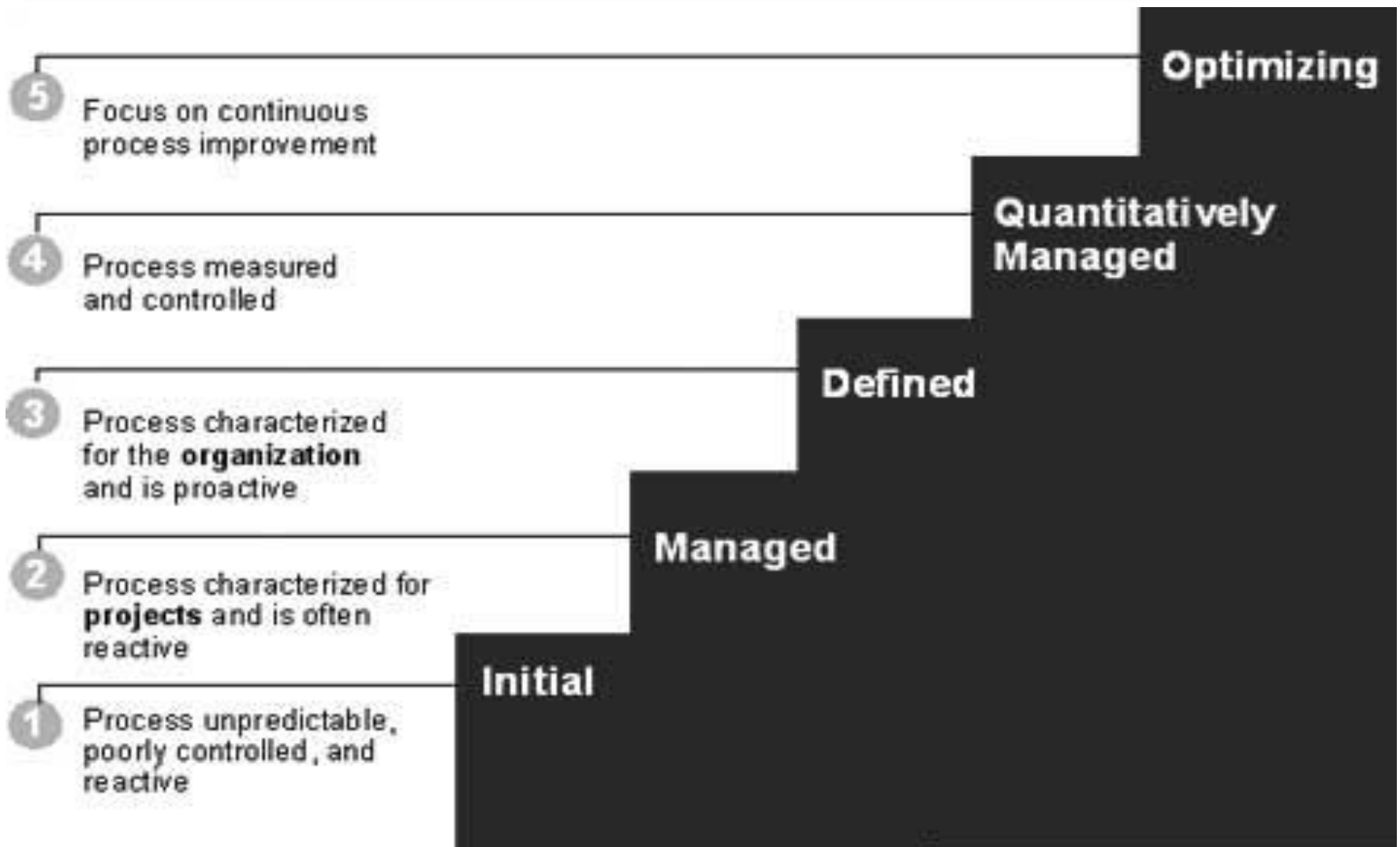


# Process measurement



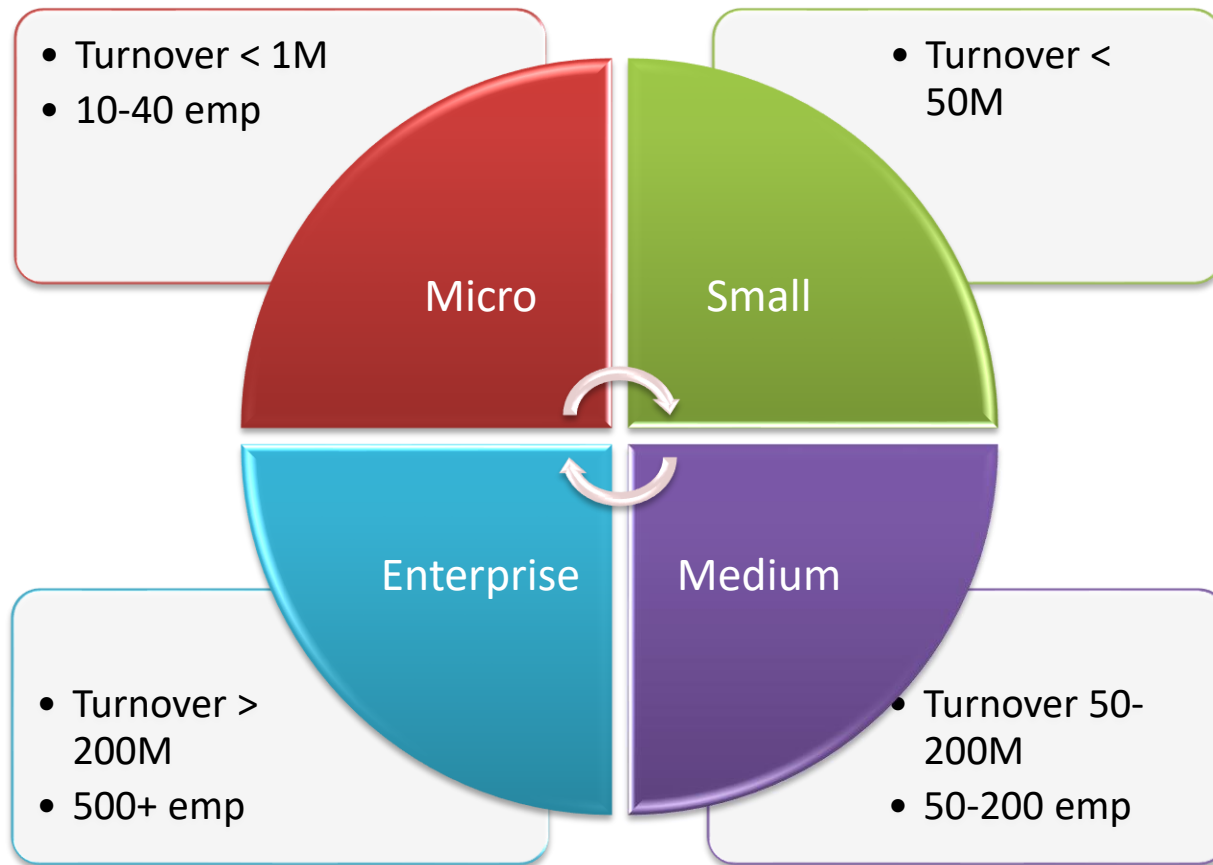
- ✧ Wherever possible, **quantitative** process data should be collected
  - However, where organisations do not have clearly defined process standards this is very difficult as you don't know what to measure. A process may have to be defined before any measurement is possible.
- ✧ Process measurements should be used to assess process improvements
  - But this does not mean that measurements should drive the improvements. The improvement driver should be the organizational objectives.

# Capability maturity levels



| Level                     | Focus                          | Key Process Area                                                                                                                                                                                                                                                           | Result                        |
|---------------------------|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|
| 1. Initial                | Process is informal and Adhoc  |                                                                                                                                                                                                                                                                            | Lowest Quality / Highest Risk |
| 2. Managed                | Basic Project Management       | <ul style="list-style-type: none"> <li>❖ Requirements Management</li> <li>❖ Project Planning</li> <li>❖ Project Monitoring and Control</li> <li>❖ Measurement and Analysis</li> <li>❖ Process and Product Quality Assurance</li> <li>❖ Configuration Management</li> </ul> | Low Quality / High Risk       |
| 3. Defined                | Process Standardization        | <ul style="list-style-type: none"> <li>❖ Requirements Development</li> <li>❖ Technical Solution</li> <li>❖ Verification</li> <li>❖ Validation</li> <li>❖ Organizational Training</li> <li>❖ Risk Management</li> </ul>                                                     | Medium Quality / Medium Risk  |
| 4. Quantitatively Managed | Quantitatively Managed         | <ul style="list-style-type: none"> <li>❖ Organizational Process Performance</li> <li>❖ Quantitative Project Management</li> </ul>                                                                                                                                          | Higher Quality /Lower Risk    |
| 5. Optimizing             | Continuous Process Improvement | <ul style="list-style-type: none"> <li>❖ Organizational Innovation and Deployment</li> <li>❖ Causal Analysis and Resolution</li> </ul>                                                                                                                                     | Highest Quality / Lowest Risk |

# Business activity categorization



# ITIL (IT Infrastructure Library)



- A set of best practices for IT service management (ITSM).
- Focuses on delivering value to customers through IT services.
- Provides a structured approach to managing the entire IT service lifecycle.
- Includes the process of managing the lifecycle of IT assets (ITAM) throughout their acquisition, deployment, utilization, and disposal.

# Enterprise Architecture, ToGAF

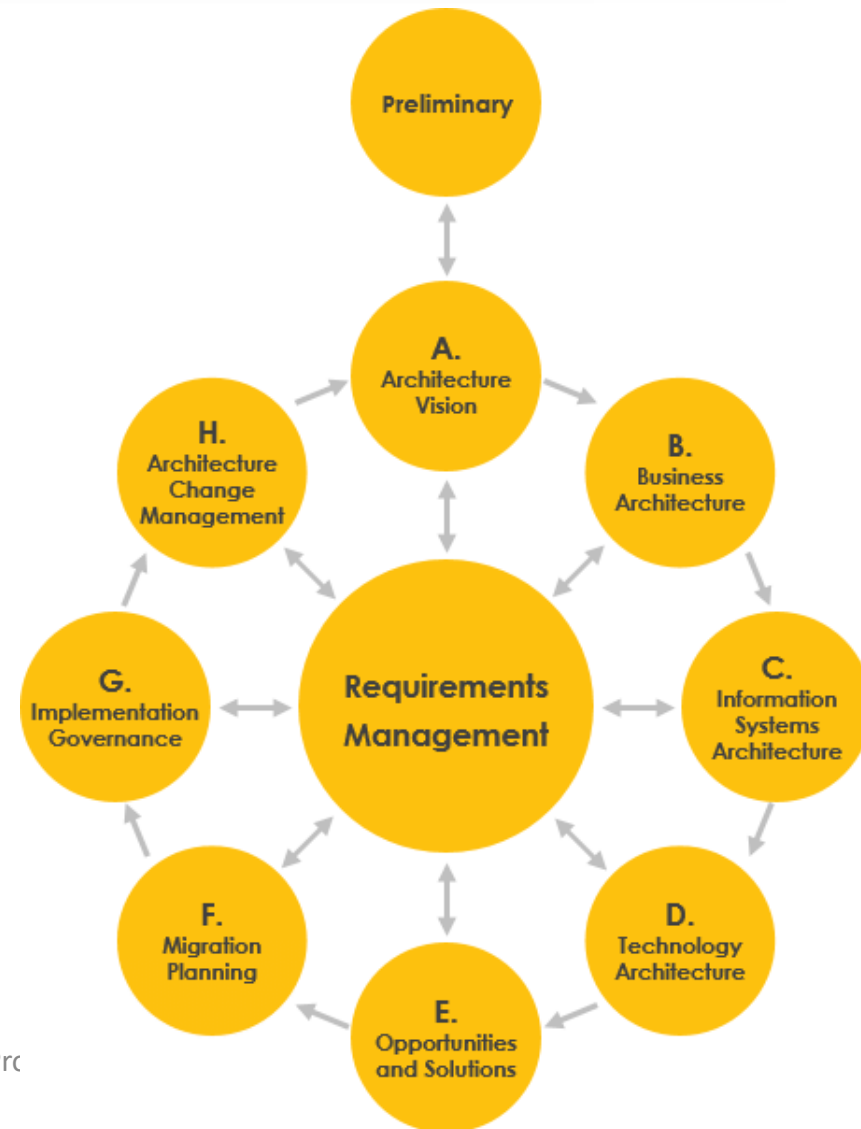


- Organizations are complex systems with interconnected elements like business processes, information systems, and technology.
- Enterprise Architecture (EA) provides a unified view of these elements, showing their relationships and how they support the organization's strategy.
- Without EA, managing this complexity can be challenging, leading to inefficiencies, misalignment, and missed opportunities.

# TOGAF (The Open Group Architecture Framework)



- A widely adopted framework for developing and managing enterprise architecture
- It offers a structured approach through its Architecture Development method (ADM)



# TOGAF phases



1. **Vision:** Defines the overall vision and goals for the enterprise architecture.
2. **Business Architecture:** Analyzes business processes, capabilities, and information needs.
3. **Information Systems Architecture:** Defines the architecture of information systems and data.
4. **Technology Architecture:** Describes the infrastructure and technology components.
5. **Opportunities & Solutions:** Identifies potential solutions and opportunities.
6. **Requirements Management:** Defines and manages requirements for the architecture.
7. **Implementation Governance:** Establishes processes for implementing and overseeing changes.
8. **Architecture Change Management:** Controls and manages changes to the architecture.



# Questions

---



1. What is the software process? what is software process model?
2. Mention some perspectives for the software process model
3. What are the categories of software process?
4. Practically, do you think that Agile is the mostly widely used process type?
5. Mention some of software process models
6. Based on your understanding, what are the common phases of waterfall process model?

# Questions

---



7. Do you think that waterfall model are no longer used? why?
8. What is incremental development model?
9. Incremental software development could be very effectively used for customers who do not have a clear idea about the systems needed for their operations. Discuss!
10. What are the possible activities of any software process model?
11. Based on your understanding, what is requirements elicitation and analysis? why it's so important in the requirement engineering process?

# Questions

---



12. What are the possible documents produced from the requirement engineering process?
13. Why it is important to make a distinction between developing the user requirements and developing system requirements in the requirements engineering process?
14. What are the common design activities in software design and implementation activity?
15. Based on your understanding, mention the difference between software verification and validation?
16. Explain why software testing should always be an incremental, staged activity?

# Questions

---



17. Are programmers the best people to test the programs that they have developed? Discuss!
18. What is the difference between component testing, system testing and UAT?
19. Based on your understanding about software evolution and requirement changes, how do you anticipate/tolerate with changes? Discuss
20. Based on your understanding, what is software prototype? why is it used? What are the possible uses for it?
21. Based on your understanding, what are the possible steps/process you would follow to develop a prototype?

# Questions

---



You have developed a prototype of a software system and your manager is very impressed by it. He proposes that it should be put into use as a production system!, with new features added as required. This avoids the expense of system development and makes the system immediately useful. Discuss with your manager explaining why prototype systems should not normally be used as production systems

# Questions

---



- 22. Based on your understanding, What is incremental delivery and development? what is the pros of using it?
- 23. Based on your understanding, why we need continuous software process improvement? what is the role of SPI engineer?
- 24. What are the possible approaches for software process improvements?
- 25. What are the major difference between CMM and Agile approach for software process improvement?